
Heart Disease Risk Predictor

Presented by: Wang Haitao,
Ritu Arora

Goal of the Project

- Build a machine learning model to predict heart disease risk
- Apply data science workflow: EDA → Feature Engineering → Preprocessing → Modeling → Pipeline
- Deliverable: Full-stack app (React frontend + API backend + ML model)
- Why it matters: accessible health insights, learning full workflow

Frontend: Routes

 routes.jsx ×

src >  routes.jsx > ...

```
1  import Home from './pages/Home'
2  import Calculator from './pages/Calculator'
3  import Results from './pages/Results'
4  import Resources from './pages/Resources'
5  import Disclaimer from './pages/Disclaimer'
6
7  export default [
8    { path: '/', element: Home },
9    { path: '/calculator', element: Calculator },
10   { path: '/results', element: Results },
11   { path: '/resources', element: Resources },
12   { path: '/disclaimer', element: Disclaimer },
13 ]
14
```

Frontend: Main reused components - Field

```
Field.jsx ×
src > components > Field.jsx > ...
1  export default function Field({ label, unit, error, children }) {
2    return (
3      <div>
4        <label className="text-sm font-semibold □text-slate-800">
5          {label}
6          {unit && <span className="ml-1 text-xs □text-slate-500">({unit})</span>}
7        </label>
8        <div className="mt-1">{children}</div>
9        {error && <p className="mt-1 text-xs ■text-red-600">{error}</p>}
10      </div>
11    )
12  }
13
```

Age group

45-49

Alcohol consumption

No

Yes

Height (cm)

-1

Value must be ≥ 0

Must be ≥ 0

Frontend: Main reused components - Segment

Segment.jsx

src > components > Segment.jsx > Segment

```
1 import clsx from 'clsx'
2
3 export default function Segment({ name, setValue, options, defaultIndex = 0 }) {
4   return (
5     <div className="inline-flex overflow-hidden rounded-md border border-slate-300 bg-
6       {options.map(([val, label], idx) => (
7         <button
8           key={val}
9           type="button"
10          className={clsx(
11            'px-4 py-2 text-sm font-semibold text-slate-700',
12            idx === defaultIndex && 'bg-navy text-white'
13          )}>
14            onClick={(e) => {
15              const parent = e.currentTarget.parentElement
16              [...parent.children].forEach(btn => btn.classList.remove('bg-navy', 'text-w
17              e.currentTarget.classList.add('bg-navy', 'text-white')
18              setValue(name, val, { shouldValidate: true, shouldDirty: true })
19            }}
20          >
21            {label}
22          </button>
23        )}>
24      </div>
25    )
26  }
```

Sex

Male

Female

Smoking history

No

Yes

Fried potato consumption

No

Yes

```
<Field label="Sex">
  <Segment
    name="Sex"
    setValue={setValue}
    options={[
      ['1', 'Male'],
      ['0', 'Female'],
    ]}
  />
</Field>
```

```
<Segment
  name="Smoking_History"
  setValue={setValue}
  options={[
    ['0', 'No'],
    ['1', 'Yes'],
  ]}
/>
```

Frontend: Data model and validation

```
const HEALTH_OPTIONS = ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']
const AGE_GROUPS = [
  '18-24',
  '24-29',
  '30-34',
  '35-39',
  '40-44',
  '45-49',
  '50-54',
  '55-59',
  '60-64',
  '65-69',
  '70-74',
  '75-79',
  '80+',
]
```

```
const schema = z.object({
  General_Health: z.enum(HEALTH_OPTIONS),
  Checkup: z.enum(['0', '1']),
  Exercise: z.enum(['0', '1']),
  Skin_Cancer: z.enum(['0', '1']),
  Other_Cancer: z.enum(['0', '1']),
  Depression: z.enum(['0', '1']),
  Diabetes: z.enum(['0', '1']),
  Arthritis: z.enum(['0', '1']),
  Sex: z.enum(['0', '1']),
  Age_Category: z.enum(AGE_GROUPS),

  Height_cm: z.coerce.number().min(0, { message: 'Must be ≥ 0' }),
  Weight_kg: z.coerce.number().min(0, { message: 'Must be ≥ 0' }),

  BMI: z.coerce.number().min(10, { message: 'Min 10' }).max(60, { message: 'Max 60' }),

  Smoking_History: z.enum(['0', '1']),
  Alcohol_Consumption: z.enum(['0', '1']),
  Fruit_Consumption: z.enum(['0', '1']),
  Green_Vegetables_Consumption: z.enum(['0', '1']),
  FriedPotato_Consumption: z.enum(['0', '1']),
})
```

ML: Dataset & Target

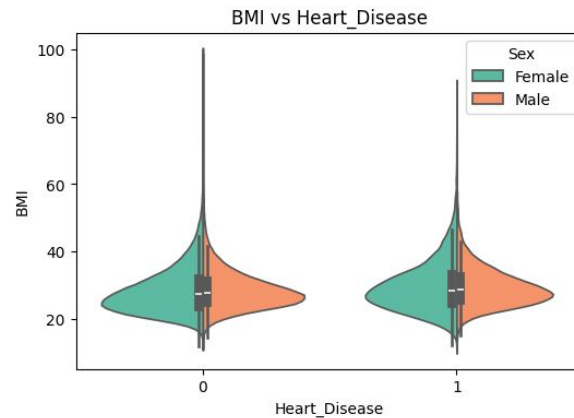
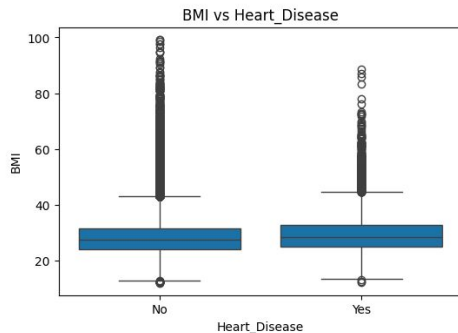
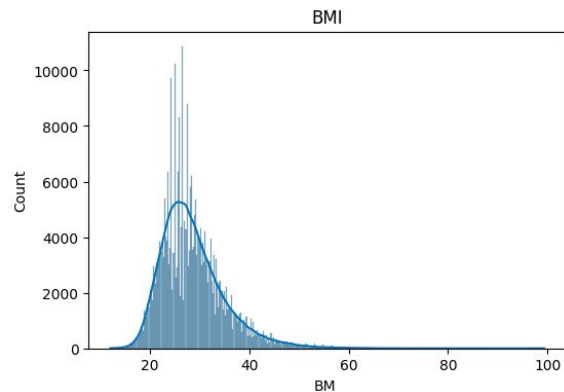
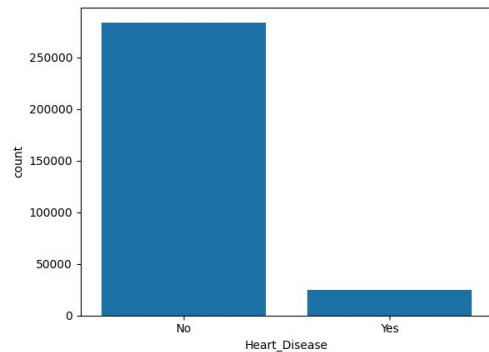
Dataset source ([Cardiovascular Diseases Risk Prediction Dataset - Gigasheet](#))

Target variable: Heart_Disease (Yes/No)

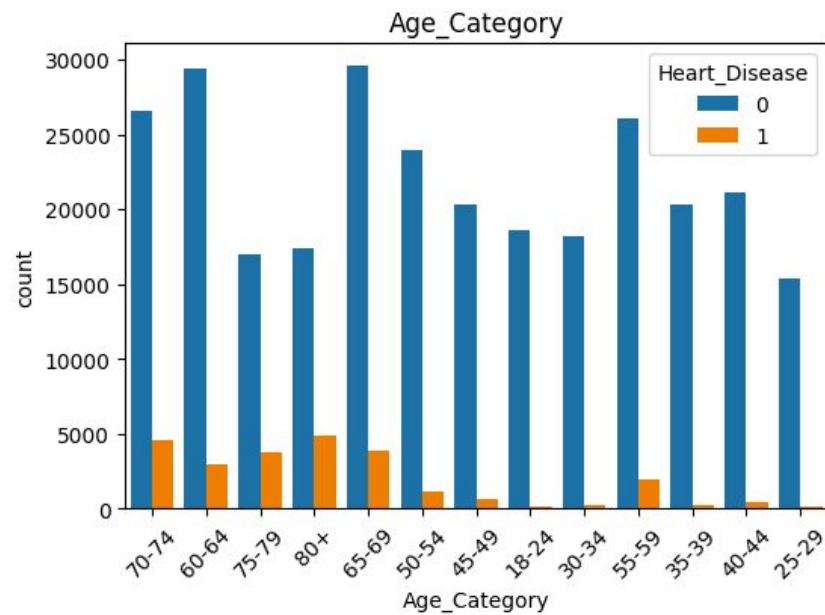
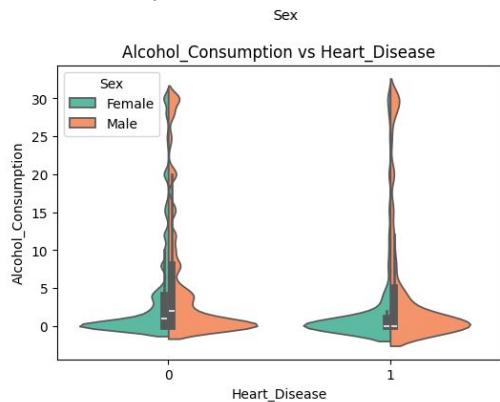
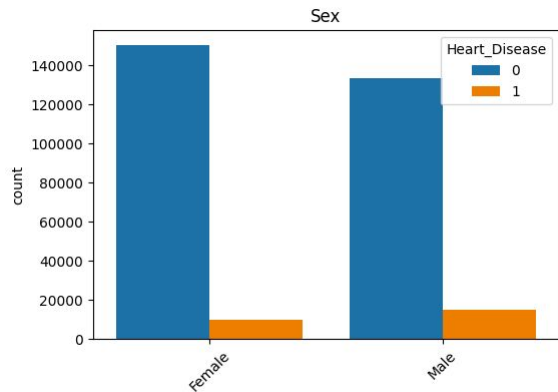
Dataset size (#rows = 308854, #features=19)

```
Index(['General_Health', 'Checkup', 'Exercise', 'Heart_Disease', 'Skin_Cancer',  
      'Other_Cancer', 'Depression', 'Diabetes', 'Arthritis', 'Sex',  
      'Age_Category', 'Height_(cm)', 'Weight_(kg)', 'BMI', 'Smoking_History',  
      'Alcohol_Consumption', 'Fruit_Consumption',  
      'Green_Vegetables_Consumption', 'FriedPotato_Consumption'],  
      dtype='object')
```

Exploratory Data Analysis (EDA)



EDA (contd..)



EDA (contd..)

	Feature	Max_Rate	Min_Rate	Range	Predictive?
7	Heart_Disease	1.000000	0.000000	1.000000	Yes
8	General_Health	0.317889	0.019927	0.297962	Yes
11	Age_Category	0.218041	0.005032	0.213010	Yes
4	Diabetes	0.208509	0.036281	0.172227	Yes
5	Arthritis	0.141010	0.051587	0.089422	Maybe
2	Other_Cancer	0.157808	0.072608	0.085200	Maybe
1	Skin_Cancer	0.156365	0.072728	0.083636	Maybe
9	Checkup	0.094560	0.025482	0.069078	Maybe
0	Exercise	0.129604	0.066701	0.062903	Maybe
6	Smoking_History	0.116426	0.056577	0.059849	Maybe
10	Sex	0.101394	0.061787	0.039607	Maybe
3	Depression	0.098561	0.076411	0.022149	Maybe

EDA (contd..)

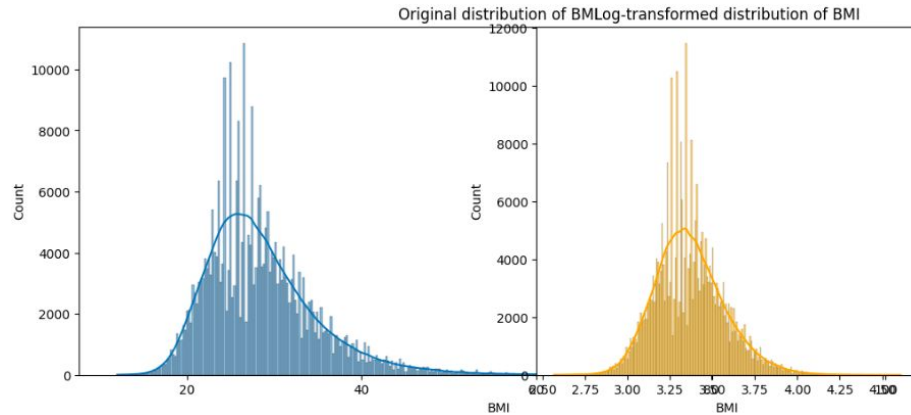
EDA Summary Table:

Feature	Skewness	Median_Diff	Outliers	Predictive?
Height_(cm)	0.02	3.00	1055	Maybe
Weight_(kg)	1.06	2.72	7326	Yes
BMI	1.38	1.11	9530	Yes
Alcohol_Consumption	1.89	1.00	36147	Yes
Fruit_Consumption	1.25	0.00	68225	Maybe
Green_Vegetables_Consumption	2.42	2.00	11878	Yes
FriedPotato_Consumption	4.91	0.00	19456	Maybe

Feature Engineering & Preprocessing

- Encoded categorical variables
- Scaled numerical features
- Train/test split (80/20, stratified)

Feature Engineering & Preprocessing (contd..)



Skewness before and after log transform:

Height(cm): Before=0.02, After=-0.20

Weight(kg): Before=1.06, After=0.18

BMI: Before=1.38, After=0.48

Alcohol_Consumption: Before=1.89, After=0.65

Fruit_Consumption: Before=1.25, After=-0.87

Green_Vegetables_Consumption: Before=2.42, After=-0.62

FriedPotato_Consumption: Before=4.91, After=0.03

Feature Engineering & Preprocessing (contd..)

```
# Implementing ordinal encoding for other multi-categorical columns
from sklearn.preprocessing import OrdinalEncoder

# Define custom orderings for each ordinal feature
ordinal_mapping={
    'General_Health': ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent'],
    'Checkup': ['Never', '5 or more years ago', 'Within the past 5 years',
               'Within the past 2 years', 'Within the past year'],
    'Age_Category': ['18-24', '25-29', '30-34', '35-39', '40-44',
                    '45-49', '50-54', '55-59', '60-64', '65-69',
                    '70-74', '75-79', '80+']
}

# Columns to encode
ordinal_cols=list(ordinal_mapping.keys())

# Create encoder with specific category orders
encoder = OrdinalEncoder(categories=[ordinal_mapping[col] for col in ordinal_cols])

# Fit and transform
df_scaled[ordinal_cols] = encoder.fit_transform(df_scaled[ordinal_cols])
```

Feature Engineering & Preprocessing (contd..)

```
print(df_scaled.dtypes)
```

General_Health	float64
Checkup	float64
Exercise	int64
Heart_Disease	int64
Skin_Cancer	int64
Other_Cancer	int64
Depression	int64
Diabetes	object
Arthritis	int64
Sex	int64
Age_Category	float64
Height_(cm)	float64
Weight_(kg)	float64
BMI	float64
Smoking_History	int64
Alcohol_Consumption	float64
Fruit_Consumption	float64
Green_Vegetables_Consumption	float64
FriedPotato_Consumption	float64
Weight_(kg)_log	float64
BMI_log	float64
Alcohol_Consumption_log	float64
Fruit_Consumption_log	float64
Green_Vegetables_Consumption_log	float64
FriedPotato_Consumption_log	float64
dtype:	object

Model Training & Evaluation

Models tested: Logistic Regression, Random Forest, XGBoost

Metrics: Accuracy, Precision, Recall, F1, ROC-AUC

Medical relevance: Recall/F1 are more important than accuracy

Model Training & Evaluation (contd..)

Model	Precision (1)	Recall (1)	F1 (1)	Accuracy	ROC AUC	Notes
1. Logistic Regression	0.21	0.80	0.33	74%	0.839	Strong recall, weak precision
2. Random Forest	0.48	0.02	0.04	92%	0.816	Severe underfitting on class 1
3. Random Forest (Tuned)	–	–	0.35	–	–	Threshold tuned to F1 = 0.35
4. XGBoost (default)	0.21	0.78	0.33	74%	0.832	Nearly identical to Logistic Regression
5. XGBoost (tuned)	0.285	0.501	0.363	86%	0.832	Balanced precision/recall after tuning

Model Training & Evaluation (contd..)

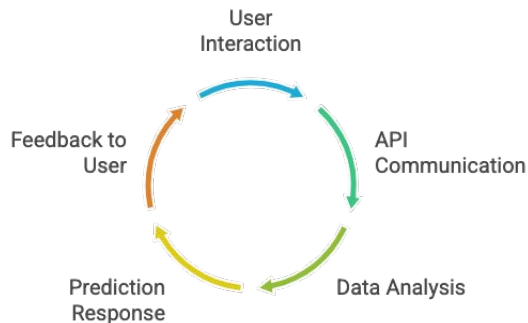
Metric	Winner	Reason
Highest Recall	Logistic / XGB (default)	~78–80% recall, best for catching positives
Highest Precision	Random Forest (default)	0.48 — but useless recall
Best F1 (class 1)	XGBoost (tuned)	0.363 — best precision-recall balance
Best Accuracy	XGBoost (tuned)	86%
Best AUC	Logistic Regression	0.839 — slightly better ranking ability

Goal	Best Model
We want to flag high-risk patients while minimizing false positives (trust)	XGBoost with tuned threshold (0.715)
We want to catch as many heart disease cases as possible, and false positives are OK	Logistic Regression or XGBoost (default threshold)
We want a balance of both	XGBoost (tuned) — F1 of 0.363 and good accuracy

Full-Stack Integration Plan

- **Backend:** FastAPI serving predictions
- **Frontend:** React app where users input health parameters and get predictions
- **Deployment:** Docker / cloud hosting (future step)
- *React UI → API → ML Model → Prediction Response*

Data Processing and Prediction Cycle



Thank you

Confusion matrix
[[41525 15252]
[986 4008]]

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.73	0.84	56777
1	0.21	0.80	0.33	4994
accuracy			0.74	61771
macro avg	0.59	0.77	0.58	61771
weighted avg	0.91	0.74	0.80	61771

ROC AUC Score: 0.8394

Type	Count	Meaning
True Negatives (TN)	41525	Correctly predicted no heart disease
False Positives (FP)	15252	Incorrectly predicted heart disease
False Negatives (FN)	986	Missed actual heart disease cases

| True Positives (TP) | 4008 | Correctly predicted heart disease

Metric	Value	Interpretation
Precision	0.21	Only 21% of predicted heart disease cases are correct
Recall	0.80	Great! 80% of actual heart disease cases are caught
F1-score	0.33	Low → due to low precision

Catching most heart disease cases (high recall) Very good AUC → model is learning the difference well

Problems:

Precision is low (21%) → Many false alarms (predicting disease when there's none) F1-score is low due to the imbalance between precision and recall

This is not ideal in a real-world medical setting — too many false positives can overwhelm the system or alarm patients unnecessarily.

```
Confusion matrix
[[56653  124]
 [ 4880  114]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.92	1.00	0.96	56777
1	0.48	0.02	0.04	4994
accuracy			0.92	61771
macro avg	0.70	0.51	0.50	61771
weighted avg	0.88	0.92	0.88	61771

ROC AUC Score: 0.8155

Metric	Value	Meaning
True Negatives (TN)	56,653	Predicted no disease correctly
False Positives (FP)	124	Predicted disease incorrectly
False Negatives (FN)	4,880	Missed heart disease cases (very bad)
True Positives (TP)	114	Predicted disease correctly
Metric	Value	Notes
-----	-----	-----
Precision	0.48	Almost half of predicted heart disease cases are correct — okay-ish
Recall	0.02	Only 2% of actual heart disease cases are detected — very bad
F1-score	0.04	Extremely low overall effectiveness for class 1

ROC AUC = 0.8155 Slightly worse than logistic regression (which had 0.839)

Tells us: the model has learned some signal, but the threshold is the problem

What Went Wrong? Random Forest is: Very conservative → it almost always predicts "no heart disease" (class 0) Why? Because 92% of your data is class 0 — the model is overwhelmed by the majority It minimizes total error but sacrifices the minority class Even though you used `class_weight='balanced'`, that alone isn't enough.

Confusion Matrix at Optimal Threshold:

```
[[49520  7257]
```

```
 [ 2433  2561]]
```

Classification Report at Optimal Threshold:

	precision	recall	f1-score	support
0	0.95	0.87	0.91	56777
1	0.26	0.51	0.35	4994
accuracy			0.84	61771
macro avg	0.61	0.69	0.63	61771
weighted avg	0.90	0.84	0.87	61771

Confusion Matrix:

```
[[41877 14900]
```

```
 [ 1121  3873]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.74	0.84	56777
1	0.21	0.78	0.33	4994
accuracy			0.74	61771
macro avg	0.59	0.76	0.58	61771
weighted avg	0.91	0.74	0.80	61771

ROC AUC Score: 0.8323512176779915

Confusion Matrix with Best Threshold:

```
[[50510  6267]
```

```
 [ 2493  2501]]
```

Classification Report with Best Threshold:

	precision	recall	f1-score
0	0.95	0.89	0.92
1	0.29	0.50	0.36
accuracy			0.86
macro avg	0.62	0.70	0.64
weighted avg	0.90	0.86	0.88

◆ Algorithms

1. Logistic Regression

- Model predicts probability of class 1:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

- Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) \geq \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

2. Random Forest

- An **ensemble** of T decision trees.
- Prediction (classification):

$$\hat{y} = \text{mode}\{\hat{y}^{(1)}, \hat{y}^{(2)}, \dots, \hat{y}^{(T)}\}$$

where $\hat{y}^{(t)}$ is the prediction of the t -th tree.

- Each tree is trained on a **bootstrap sample** of the data, and splits are made using criteria like **Gini impurity**:

$$Gini(S) = 1 - \sum_{i=1}^C p_i^2$$

(where p_i is the fraction of samples of class i).

3. XGBoost (Extreme Gradient Boosting)

- Builds trees **sequentially**, each correcting errors of the previous ones.
- Objective function:

$$\text{Obj}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where

- l = loss function (e.g., logistic loss),
- $\Omega(f_k)$ = regularization term (controls complexity),
- f_k = decision tree.
- Logistic loss:

$$l(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

1. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision

(How many predicted positives are actually correct?)

$$Precision = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity, True Positive Rate)

(How many actual positives did we catch?)

$$Recall = \frac{TP}{TP + FN}$$

4. F1 Score

(Harmonic mean of precision and recall)

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

5. ROC AUC (Area Under the Receiver Operating Characteristic Curve)

- ROC curve plots TPR vs FPR at all thresholds.
-

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

- AUC = area under this curve, ranges from 0.5 (random guess) → 1.0 (perfect).

True Positive Rate (TPR) = Recall / Sensitivity

👉 Measures how many actual positives were caught.

$$TPR = \frac{TP}{TP + FN}$$

- High TPR means the model catches most patients who truly have heart disease.
 - On ROC curve → y-axis.
-

False Positive Rate (FPR)

👉 Measures how many actual negatives were incorrectly flagged positive.

$$FPR = \frac{FP}{FP + TN}$$

- High FPR means too many false alarms (healthy patients incorrectly flagged).
 - On ROC curve → x-axis.
-

ROC Curve

- Plot TPR (y-axis) vs FPR (x-axis) at different thresholds.
- A good model's curve **hugs the top-left corner** (high TPR, low FPR).
- **AUC (Area Under Curve):**
 - 0.5 → Random guessing.
 - 0.7–0.8 → Acceptable.
 - 0.8–0.9 → Good.
 - 0.9 → Excellent.

ROC Curves Example

